

```

    }
}
}
return 0;
} 여기서 파티를 만들고 싶은데 배열과 포인터를 쓰면 어떻게 만들 수 있어?

```



C언어로 만든 RPG 게임 코드, 구조가 아주 깔끔하고 재밌네요! 캐릭터 생성, 사냥터, 보스 레이드까지 기본 루프가 잘 짜여 있습니다.

여기에 배열(Array)과 포인터(Pointer)를 사용해 '파티 시스템'을 도입하려면, 기존의 단일 변수들(`name`, `job`, `hp` 등)을 여러 개 저장할 수 있는 공간으로 확장해야 합니다.

가장 깔끔하고 효율적인 방법은 `struct` (구조체)를 정의하고, 이 구조체의 배열과 포인터를 사용하는 것입니다. 구조체를 쓰지 않으면 파티원 3명의 이름을 저장하기 위해 `name1`, `name2`, `name3` 처럼 변수가 지저분해지거든요.

기존 코드를 바탕으로 3인 파티 시스템을 구현하는 핵심 아이디어와 수정된 전체 코드를 소개해 드릴게요.

💡 파티 시스템 구현 핵심 아이디어

- 1 **캐릭터 구조체 생성:** 캐릭터의 이름, 직업, 레벨, 스탯을 하나로 묶는 `Character` 구조체를 만듭니다.
- 2 **구조체 배열:** `Character party[3];` 을 선언하여 최대 3명의 파티원을 저장합니다.
- 3 **포인터 활용:** 함수에 파티원 정보를 넘겨주거나 제어할 때 구조체 포인터(`Character*`)를 사용해 메

+ Gemini에게 물어보기

Flash ▾



🔧 주요 변경점 및 포인터/배열 설명

1. `struct` 와 배열의 만남

C

```
Character party[3];
```

기존에는 캐릭터 1명의 정보만 다뤘지만, 이제 `party` 라는 이름의 배열을 만들어 최대 3명의 구조체 데이터를 통째로 관리합니다.

2. 화살표 연산자(`->`)와 포인터

함수나 메인 루프 안에서 구조체 포인터를 사용할 때, `.` 대신 `->` 연산자를 사용합니다.

C

```
Character* new_char = &party[member_count];  
new_char->hp = 100; // (*new_char).hp = 100; 과 같은 의미입니다.
```

`update_power(Character* c)` 함수처럼 포인터를 매개변수로 받으면, 메인 함수에 있는 원본 데이터의 값을 직접 수정할 수 있어 복사 비용이 들지 않고 효율적입니다.

3. 포인터 연산을 통한 배열 순회

`print_party_status` 함수를 보면 배열을 포인터처럼 다루는 문법을 볼 수 있습니다.

C

```
Character* c = party + i;
```

C언어에서 배열의 이름(`party`)은 배열 첫 번째 요소의 주소값입니다. 여기에 `+ i` 를 하면 `i` 번째 요소의 주소로 이동하게 됩니다. 배열과 포인터가 본질적으로 같다는 점을 활용한 방식입니다.

캐릭터 생성과정에서 for문을 안써도 돼?

네, 캐릭터를 생성할 때는 `for` 문을 쓰지 않는 것이 오히려 맞습니다! `for` 문은 보통 무언가를 '여러 번 반복'할 때 사용하죠. 하지만 캐릭터 생성 메뉴(1번 메뉴)는 사용자가 키보드로 '방금 생성할 캐릭터 1명의 정보'만 딱 한 번 입력하는 곳이기 때문에 `for` 문을 쓸 필요가 없습니다.

대신, 파티원들이 동시에 다 같이 행동해야 하는 2번(사냥터)과 3번(보스 레이드) 과정에서는 `for` 문이 반드시 필요합니다.

여기서 `for` 문이 쓰이고 안 쓰이는지 명확하게 정리해 드릴게요.

1. 1번 메뉴 (캐릭터 생성): `for` 문 필요 없음 ❌

사용자가 캐릭터 생성 메뉴를 누를 때마다 1명씩만 차례대로 추가하면 됩니다.

이때는 `for` 문 대신, 우리가 가지고 있는 `member_count` (현재 파티원 수) 변수가 알아서 방 번호 역할을 해주기 때문에 한 번만 입력받고 끝냅니다.

- 첫 번째 실행: `member_count` 가 0이므로 `name[0]` 에 저장 후 `member_count` 는 1이 됨.
- 두 번째 실행: `member_count` 가 1이므로 `name[1]` 에 저장 후 `member_count` 는 2가 됨.

2. 2번 메뉴 (사냥터) & 3번 메뉴 (보스): `for` 문 필수 ○

여기서는 이미 만들어진 파티원 전체(여러 명)를 한꺼번에 처리해야 하므로 `for` 문이 꼭 필요합니다.

- 사냥터: 파티원 0번부터 `member_count - 1` 번까지 모든 대원의 경험치와 레벨을 동시에 올려야 하므로 `for` 문을 돌립니다.
- 보스 레이드: 파티원 모두의 전투력을 하나로 더해서 합산해야 하므로 `for` 문으로 배열을 처음부터 끝까지 훑어야 합니다.

💡 한눈에 보는 구조 차이

- 캐릭터 생성시: `member_count` 가 가리키는 딱 한 칸에만 데이터를 콕 집어 넣음 (반복 없음).
- 사냥/보스 레이드시: `for(i = 0; i < member_count; i++)` 를 이용해 0번 칸부터 저장된 칸까지 주르륵 훑으며 지나감.

결론적으로 형찬님이 작성하신 현재 코드 구조가 가장 정석적이고 올바른 방법입니다! 생성할 때는 `for` 문 없이 `member_count` 로만 방을 지정하고, 사냥과 전투를 할 때만 `for` 문으로 파티원 전체를 관리하는 방식이 아주 잘 짜인 구조입니다.

♡ ♡ ♡ ...

파티원을 3명으로 만드는거면 필요한거 아니야?

아하, 어떤 의문이 드셨는지 완벽히 이해했어요! "결국 총 3명을 만들 거니까 `for` 문을 3번 돌려야 하는 거 아닌가?"라는 생각이 드시는 게 당연합니다.

결론부터 말씀드리면, `for` 문을 어디에 쓰느냐에 따라 게임의 진행 방식이 완전히 달라집니다.

1. 만약 캐릭터 생성에 `for` 문을 쓰면? (게임 시작하자마자 3명 연속 입력)

메뉴 1번을 눌렀을 때 `for` 문을 써서 3번 반복하게 만들면, 게임을 시작하자마자 사용자는 1번 대원, 2번 대원, 3번 대원의 이름과 스탯을 그 자리에서 연속으로 입력해야 합니다.

C

```
// 만약 이렇게 짜면?  
case 1:  
    for(i = 0; i < 3; i++) {  
        printf("%d번째 캐릭터 이름 입력: ", i + 1);  
        scanf("%c", &name[i]);  
        // ... 직업 및 스탯 입력 ...  
    }  
    member_count = 3;  
    break;
```

이렇게 하면 1번 메뉴를 딱 한 번만 눌러도 파티원 3명이 동시에 완성되겠죠! "나는 게임 시작할 때 한 번에 팀원 3명을 다 뽑아놓고 시작하겠다" 하시면 이 방법이 맞습니다.

2. 지금 코드 방식 (메뉴를 누를 때마다 1명씩 추가)

현재 형찬님의 코드는 `for` 문이 없는 대신, 게임 루프인 `while(1)` 이 계속 돌고 있습니다. 그래서 아래처럼 사용자가 원할 때 1명씩 끊어서 파티원을 늘려가는 방식입니다.

- 1 게임 실행 후 1번 메뉴 선택 → 1번 대원 생성 (`member_count` 가 1이 됨) → 메인 메뉴로 복귀
- 2 2번 메뉴(사냥터) 가서 1번 대원만 데리고 사냥해보기
- 3 다시 1번 메뉴 선택 → 2번 대원 생성 (`member_count` 가 2가 됨) → 메인 메뉴로 복귀

즉, 대괄호 안의 `member_count` 변수가 `for` 문의 `i` 처럼 "다음 방 번호"를 기억하고 있기 때문에, `for` 문 없이도 1번 메뉴를 누를 때마다 0번 방, 1번 방, 2번 방 순서대로 차곡차곡 쌓여서 결국 3명까지 만들어지는 구조인 거죠.

💡 요약하자면!

- "1번 메뉴 누르자마자 귀찮은 스탯 입력을 3명 연속으로 다 끝내버리고 싶다!" 하시면 `for` 문을 쓰는 게 맞습니다.
- "일단 1명만 만들어서 키우다가, 나중에 필요할 때 파티원을 하나씩 추가하고 싶다!" 하시면 지금처럼 `for` 문 없이 `member_count` 로만 관리하는 게 맞습니다!

```
}  
}  
return 0;  
} 레벨업 결과 출력에서 여러번이 반복해서 뜨는데 한번만 뜨게 할 수  
있어?  
간단히 ^
```

가능해. 현재 코드에는 사실 `printf("모든 파티원이 ...")` 가 한 번만 실행되도록 되어 있는데, 다른 문제가 있어.

```
/* C  
  
for(i=0;i<PARTY_SIZE;i++)  
{  
    int first_level = level[i];  
  
    ...  
  
    int rise_level = level[i] - first_level;  
}  
  
printf("모든 파티원이 %d만큼의 경험치를 획득해 %d레벨이 상승했습니다!\n",  
      total_gain, rise_level);
```

여기서 `rise_level` 은 `for` 문 안에서 선언된 지역변수인데, 반복문 밖에서 사용하고 있어. 원래라면 컴파일 오류가 나와 한다.

그리고 만약 각 파티원의 레벨업 결과를 출력하려고 반복문 안에 `printf()` 를 넣어두었다면 파티원 수만큼 출력될 것이다.

방법 1 : 전체 결과를 한 번만 출력

파티원들이 모두 같은 경험치를 받는 구조라면 첫 번째 파티원의 레벨 상승량만 계산해서 출력하면 된다.

```
/* C  
  
int rise_level = 0;  
  
for(i=0;i<PARTY_SIZE;i++)  
{  
    int first_level = level[i];
```

+ 무엇이든 물어보세요

방법 1 : 전체 결과를 한 번만 출력

파티원들이 모두 같은 경험치를 받는 구조라면 첫 번째 파티원의 레벨 상승량만 계산해서 출력하면 된다.

```
«» C

int rise_level = 0;

for(i=0;i<PARTY_SIZE;i++)
{
    int first_level = level[i];

    ...

    power[i] = output_power(job[i], hp[i], ad[i], ap[i]);

    if(i == 0)
        rise_level = level[i] - first_level;
}

printf("모든 파티원이 %d만큼의 경험치를 획득해 %d레벨 상승했습니다\n",
      total_gain, rise_level);
```

방법 2 : 파티원별 결과 출력 (추천)

파티원마다 현재 레벨이 다를 수 있으므로 개별적으로 출력하는 것이 더 자연스럽다.

```
«» C

for(i=0;i<PARTY_SIZE;i++)
{
    int first_level = level[i];

    ...

    power[i] = output_power(job[i], hp[i], ad[i], ap[i]);

    printf("%s : %d → %d (레벨 %d 상승)\n",
          name[i],
          first_level,
          level[i],
```

+ 무엇이든 물어보세요